

10 notions SQL que tu crois maîtriser



Celles que tout le monde cite en entretien, avec l'exemple chiffré et le piège qui testent la vraie compréhension.

Tu as écrit ces dix mots-clés des dizaines de fois. Mais réciter la définition et comprendre l'effet réel sur un chiffre, ce n'est pas la même compétence. Ce guide reprend chaque notion sur un même terrain, clients et commandes, avec l'exemple qui la rend concrète et le piège qui sépare la définition apprise de la compréhension solide.

Jointures : ce qui disparaît, ce qui explose.

Une jointure ne fait pas que rapprocher deux tables : elle peut faire disparaître des lignes, ou en faire apparaître en trop, sans aucune erreur SQL.

1. INNER JOIN vs LEFT JOIN : les lignes qui disparaissent PIÈGE CLASSIQUE

Définition : L'INNER JOIN ne garde que les lignes qui ont une correspondance des deux côtés. Le LEFT JOIN garde toutes les lignes de gauche et met NULL à droite en l'absence de correspondance.

```
SELECT c.nom, COUNT(o.id) AS nb_commandes
FROM clients c
INNER JOIN commandes o ON o.client_id = c.id
GROUP BY c.nom;
```

Exemple chiffré : sur 5 clients dont 1 sans aucune commande, cette requête ne renvoie que 4 lignes. Avec un **LEFT JOIN** à la place, les 5 lignes restent, le client sans commande affiche 0. Dès que l'absence de correspondance est une information (taux de clients actifs, churn), le LEFT JOIN est obligatoire, pas optionnel.

Piège d'entretien : « pourquoi mon taux de clients actifs dépasse 100 % ? » Le junior a compté les clients avec commande via INNER JOIN, et le total de clients via LEFT JOIN : numérateur et dénominateur ne viennent pas de la même base.

2. Les doublons cachés par une jointure (fan-out) ÉLIMINATOIRE

Définition : Une clé de jointure dupliquée côté droit multiplie les lignes de gauche : un fan-out gonfle silencieusement un agrégat sans qu'aucune ligne ne soit individuellement fausse.

```
SELECT c.id, SUM(o.montant) AS total_ventes
FROM clients c
JOIN commandes o ON o.client_id = c.id
JOIN adresses a ON a.client_id = c.id
GROUP BY c.id;
```

Exemple chiffré : le client 42 a 2 commandes à 150 € (total réel 300 €) et 3 adresses enregistrées. La double jointure produit $2 \times 3 = 6$ lignes avant l'agrégation : le SUM affiche 900 €, triplé.

Piège d'entretien : « ton chiffre d'affaires a bondi de 60 % du jour au lendemain, que vérifies-tu en premier ? » La bonne réponse commence par le grain de la jointure, pas par les ventes elles-mêmes.

Pourquoi cette question élimine

Une jointure qui multiplie les lignes peut gonfler un chiffre sans produire aucune erreur SQL.

Grain

Une ligne représente quoi?

Cardinalité

La clé est-elle unique côté droit?

Contrôle

COUNT(*) avant/après jointure?

Filter et regrouper sans te tromper.

Ici, l'ordre logique d'exécution compte plus que la syntaxe : ce que tu filtres avant l'agrégation n'est pas ce que tu filtres après.

3. WHERE vs HAVING : avant ou après l'agrégation

Définition : Le WHERE filtre les lignes brutes avant le regroupement. Le HAVING filtre les groupes après l'agrégation : il ne peut porter que sur une expression agrégée ou une colonne du GROUP BY.

```
SELECT client_id, SUM(montant) AS total_2025
FROM commandes
WHERE date_commande >= '2025-01-01'
GROUP BY client_id
HAVING SUM(montant) > 500;
```

Correction : le WHERE filtre les lignes brutes avant le regroupement (la date). Le HAVING filtre les groupes après l'agrégation (le total). Écrire WHERE SUM(montant) > 500 provoque une erreur : la fonction d'agrégation n'existe pas encore à ce stade.

Piège d'entretien : pourquoi mettre la date dans le WHERE plutôt que dans le HAVING si « ça marcherait aussi » ? Réponse attendue : le WHERE réduit le volume avant l'agrégation, donc la requête est plus rapide sur une grande table.

4. GROUP BY : ce qui doit être agrégé ou groupé ÉLIMINATOIRE

Définition : Le GROUP BY réduit un ensemble de lignes à une ligne par valeur de groupe. Toute colonne du SELECT qui n'est pas dans le GROUP BY doit passer par une fonction d'agrégation.

```
SELECT c.nom, o.produit, SUM(o.montant)
FROM clients c
JOIN commandes o ON o.client_id = c.id
GROUP BY c.nom, o.produit;
```

Correction : chaque colonne du SELECT qui n'est pas agrégée doit figurer dans le GROUP BY. Si o.produit sort du GROUP BY, PostgreSQL refuse la requête; MySQL sans ONLY_FULL_GROUP_BY l'exécute et choisit une valeur arbitraire par groupe.

Piège d'entretien : « cette requête tourne, le chiffre est-il fiable ? » Tourner sans erreur ne garantit pas un résultat qui a un sens : il faut vérifier qu'une colonne non agrégée a une seule valeur possible par groupe.

5. COUNT(*) vs COUNT(col) vs COUNT(DISTINCT) PIÈGE CLASSIQUE

Définition : COUNT(*), COUNT(colonne) et COUNT(DISTINCT colonne) répondent à trois questions différentes : combien de lignes, combien de valeurs non NULL, combien de valeurs uniques.

```
SELECT
COUNT(*)           AS lignes_totales,
COUNT(email)       AS avec_email,
COUNT(DISTINCT email) AS emails_uniques
FROM commandes;
```

Exemple chiffré : sur 10 commandes, 2 sont passées en mode invité (email NULL) et 2 emails reviennent deux fois parmi les 8 restants. Résultat : `COUNT(*) = 10`, `COUNT(email) = 8`, `COUNT(DISTINCT email) = 6`.

Piège d'entretien : « combien de clients uniques ont commandé? » Un `COUNT(client_id)` sans `DISTINCT` compte des commandes, pas des clients.

Ce que ça teste vraiment

Tu ne choisis pas une fonction, tu choisis ce que tu veux compter.

COUNT(*)

Toutes les lignes
« Combien de commandes? »

COUNT(col)

Non NULL seulement
« Combien ont un email? »

COUNT(DISTINCT col)

Valeurs uniques
« Combien de clients? »

6. ORDER BY et les agrégats

Rappel : Contrairement au `WHERE`, le `ORDER BY total DESC` s'exécute après le `SELECT` dans l'ordre logique : un alias comme `total_y` est directement utilisable, ce qui est interdit dans le `WHERE`. Trier sur une colonne qui n'est ni agrégée ni dans le `GROUP BY` échoue sur la plupart des moteurs, pour la même raison qu'une colonne non agrégée dans le `SELECT`.

NULL : la valeur qui ment.

Ici, le recruteur vérifie si tu comprends la logique à trois valeurs, ou si tu as juste mémorisé le mot NULL.

7. NULL et la logique à trois valeurs ÉLIMINATOIRE

Définition : En SQL, NULL représente une valeur inconnue. Une comparaison avec NULL ne renvoie ni VRAI ni FAUX mais UNKNOWN, une troisième valeur logique qui élimine la ligne du résultat.

```
SELECT * FROM commandes WHERE remise = NULL;
-- 0 ligne renvoyée, même si 4 lignes sur 12 ont remise = NULL

SELECT * FROM commandes WHERE remise IS NULL;
-- 4 lignes renvoyées, le résultat attendu
```

Correction : en SQL, une comparaison avec NULL ne donne jamais VRAI ni FAUX, elle donne UNKNOWN, une troisième valeur logique qui élimine la ligne du `WHERE`. Seuls `IS NULL` et `IS NOT NULL` testent NULL correctement; `=` et `<>` échouent silencieusement, sans erreur.

Piège d'entretien : « cette requête tourne sans erreur mais retourne toujours 0 ligne, pourquoi? » Cette question trie les candidats qui ont mémorisé NULL de ceux qui comprennent la logique à trois valeurs.

À retenir pour éviter le piège

NULL ne se compare pas, il se teste.

Le même piège existe avec `NOT IN` : une sous-liste contenant un seul NULL fait échouer tout le `NOT IN`, silencieusement, sans qu'aucune erreur ne le signale.

Structurer et lire une requête complexe.

Ici, le recruteur teste ta capacité à organiser une logique en plusieurs étapes, pas juste à écrire un SELECT.

8. Sous-requête vs CTE : lisibilité, réutilisation

Définition : Une sous-requête imbriquée et une CTE (`WITH`) peuvent produire exactement le même résultat : la différence n'est pas dans le résultat mais dans la lisibilité et la réutilisation.

```
WITH totaux AS (
  SELECT client_id, SUM(montant) AS total
  FROM commandes
  GROUP BY client_id
)
SELECT client_id, total
FROM totaux
WHERE total > (SELECT AVG(montant) FROM commandes);
```

Correction : une sous-requête imbriquée équivalente produit le même résultat, mais doit être recopiée si elle sert deux fois. La CTE (WITH) se lit de haut en bas comme une suite d'étapes nommées et se réutilise sans duplication.

Piège d'entretien : « pourquoi préférer une CTE ici? » Ce n'est pas une question de performance, la plupart des moteurs modernes optimisent les deux de façon équivalente : c'est une question de lisibilité, un collègue doit suivre le raisonnement sans dérouler des parenthèses imbriquées.

9. Fonctions fenêtre vs GROUP BY : garder le détail

Définition : Une fonction fenêtre (PARTITION BY) calcule une valeur par groupe sans réduire le nombre de lignes, contrairement au GROUP BY qui collapse le résultat à une ligne par groupe.

```
SELECT client_id, montant,
  ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY montant DESC) AS rang
FROM commandes;
```

Exemple chiffré : le client 42 a 3 commandes à 150 €, 90 € et 60 €. Un GROUP BY réduirait ces 3 lignes à 1 seule ligne agrégée, le détail est perdu. La fonction fenêtre garde les 3 lignes et ajoute un rang par client, sans collapser le résultat.

Piège d'entretien : « trouve la commande la plus récente de chaque client sans perdre ses autres colonnes. » Un GROUP BY + MAX(date) oblige à rejoindre la table une seconde fois; un ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date DESC) = 1 répond en une seule passe.

10. DISTINCT qui masque un vrai problème de modèle PIÈGE CLASSIQUE

Définition : DISTINCT élimine les lignes strictement identiques du résultat affiché. Il ne corrige jamais la cause d'un doublon, il la rend seulement invisible.

```
SELECT DISTINCT c.id, c.nom
FROM clients c
JOIN commandes o ON o.client_id = c.id
JOIN adresses a ON a.client_id = c.id;
```

Correction : la double jointure du chapitre 1 produit des doublons de clients; DISTINCT les fait disparaître de l'affichage, la requête a l'air propre. Mais dès qu'une colonne agrégée s'ajoute à ce SELECT, les doublons de grain reviennent fausser le calcul. DISTINCT est un pansement d'affichage, pas une correction de grain.

Piège d'entretien : « cette requête a toujours un DISTINCT au début, pourquoi? » Si personne ne sait répondre, c'est le signal qu'un problème de jointure a été maquillé plutôt que corrigé, et qu'il va réapparaître ailleurs dans le modèle.

10 sur 10. Ces notions ne se retiennent pas par cœur, elles se retiennent par le réflexe qu'elles déclenchent : vérifier le grain avant de faire confiance à une jointure, distinguer avant et après l'agrégation, tester NULL au lieu de le comparer, et ne jamais laisser DISTINCT maquiller un problème de modèle. C'est ce réflexe, pas la définition récitée, qui se voit en entretien.

Fait main, en solo.

Si cette fiche t'a aidé, garde-la sous la main pour ton prochain entretien ou ton prochain code review, et partage-la à quelqu'un qui prépare une reconversion data. Dis-moi en commentaire quelle notion tu veux voir développée ensuite.

Dylan

